

TEMA 1 – Selección de Herramientas

🎯 Objetivo del tema

En este tema aprenderemos las bases para el desarrollo de aplicaciones web en el lado del cliente

1.1 Introducción

En este primer capítulo introductorio, vamos a empezar a dar nuestros primeros pasos en el desarrollo de aplicaciones web en lado del cliente. Nuestros objetivos son:

- Conocer el **rol del desarrollo** en el lado cliente dentro de una aplicación web.
- Configurar el **entorno de trabajo** (VS Code, Node.js y Git)
- Comprender el **flujo básico de trabajo** de un proyecto web.
- Utilizar las **DevTools** del navegador para inspeccionar, depurar y medir rendimiento/accesibilidad.
- Realizar un primer ejercicio con JavaScript moderno (ES).

1.2 El Rol del Desarrollo en el Lado Cliente

Entendemos el **Desarrollo en el Entorno Cliente (*Front-end Web Development*)** como el área de la ingeniería de software web encargada de diseñar, construir e implementar la lógica de procesamiento, la interfaz gráfica y la experiencia de usuario que se ejecutan **directamente en el agente de usuario (navegador web)**, utilizando los estándares abiertos de la W3C y ECMAScript (HTML, CSS y JavaScript).

Esta es una definición estricta que te puede dar ChatGPT.

Vamos a desgranar poco a poco lo que nos están contando. En principio, todos los programas necesitan un **Entorno de Ejecución**. Esto es, un 'lugar' donde se ejecuta. Si hablamos de Desarrollo en Entorno Servidor, (*Back-end Web Development*) estaríamos hablando de un entorno controlado por una empresa (Servidores Linux, Docker...). Pero, en nuestro caso, el código **se ejecuta** en el navegador web (Chrome, Firefox...)

Este detalle es muy importante.

En general, las aplicaciones web responden al Cliente (navegador web) enviándole de vuelta ficheros de **texto plano**. El Cliente descarga, interpreta y compila en tiempo real estos ficheros para mostrarnos una página web.

Estos ficheros de texto plano son:

- **HTML (Estructura y Semántica):** Define el árbol de contenido (DOM) y qué elementos existen: botones, tablas, formularios...
- **CSS (Presentación y Diseño Adaptativo):** Define cómo se ven esos elementos: maquetación, colores, tipografía y la adaptación a distintas pantallas (*Responsive Web Design*).
- **JavaScript (Comportamiento y Lógica):** Nos permite modificar el HTML y el CSS en tiempo real, procesa datos, capturar la interacción del usuario y se comunica con el servidor en segundo plano.

1.3 La Arquitectura Web

Ya hemos aprendido que nuestro módulo se centra en código que se ejecuta exclusivamente en el Cliente, que, en nuestro caso, es un **Navegador Web** (Chrome, Firefox...)

El **Navegador Web** parece un programa sencillo, pero en realidad es un **entorno de ejecución complejo**. Ya has tenido contacto con él en módulos anteriores, como en Lenguaje de Marcas. Allí aprendiste que un sencillo fichero escrito con etiquetas HTML se renderizaba en el navegador como una serie de tablas, etiquetas, etc. Y lo mismo sucedía con el CSS y los colores, tamaños de fuente, etc.

Los componentes de un Navegador son:

- **Interfaz de Usuario (UI):** La barra de direcciones, botones de navegación, marcadores y menús. Todo lo que rodea a la ventana del documento.
- **Motor del Navegador (*Browser Engine*):** Gestiona las acciones entre la UI y el motor de renderizado.
- **Motor de Renderizado (*Rendering Engine*):** Transforma código HTML y CSS en la representación visual en pantalla.
- **Motor de JavaScript (*JS Engine*):** Interpreta, compila y ejecuta el código JavaScript.
- **Módulo de Red (*Networking*):** Gestiona las comunicaciones HTTP/HTTPS, conectividad de red, resolución DNS y caché.
- **Backend de Interfaz (*Display Backend*):** Utilizado para dibujar elementos básicos como ventanas y combos.
- **Almacenamiento de Datos (*Persistence / Storage*):** Capa de almacenamiento local en el disco del cliente.

¿Y todos los navegadores funcionan igual? Pues no, no funcionan exactamente igual.

Hubo una época, la del **Internet Explorer 6**, en la que ocurrió la Guerra de Navegadores. Microsoft tenía un monopolio (+90% de cuota) e ignoraba deliberadamente los estándares de la W3C. Como resultado, cada navegador interpretaba el CSS a su manera, se inventaba etiquetas HTML, etc. Esto hacía que los todos los desarrolladores tuviesen que tirar de 'trucos' para conseguir que una web se viese igual en todos los posibles Navegadores del mercado.

Hoy en día esto no pasa. Todo el mundo cumple los estándares de la W3C (para HTML/CSS) y de Ecma International (para JavaScript con ECMAScript).

Lo que siguen siendo diferentes son los **Motores**. Google Chrome, MS Edge, Brave y Opera usan el mismo, **Chromium**. Los otros dos importantes son minoritarios, y pertenecen a Safari y Gecko.

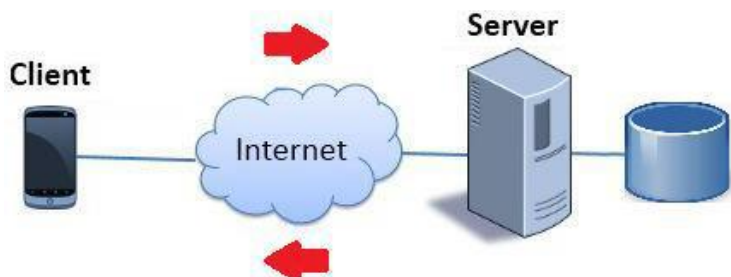
1.4 Ciclo Petición–Respuesta (HTTP)

¿Cómo se comunica un Cliente (Navegador Web) y un Servidor?

HTTP es un protocolo orientado a transacciones que sigue el esquema petición-respuesta entre un Cliente y un Servidor. El cliente (Navegador Web) realiza una petición enviando un mensaje con cierto formato al servidor, que envía siempre un mensaje de respuesta.

Estos mensajes **HTTP** son texto plano.

Y lo has adivinado: las **respuestas** son HTML, CSS y JavaScript (para este módulo).



A nivel básico, cuando escribes en un navegador una URL, sucede lo siguiente:

1. Escribes una URL y le das a aceptar: `http://www.elorrieta.eus`
2. El navegador realiza una **petición** al Servidor correspondiente de Internet
3. El servidor responde enviándonos de vuelta:
 - a. Un **código de estado**
 - b. Un fichero **HTML**
 - c. Un fichero **CSS** (normalmente)
 - d. Un **JavaScript** (no todas las páginas tienen).
4. El navegador renderiza estas tres partes para mostrarte el contenido de la web.

🔔 Importante – ¿Envía tres ficheros de verdad?

Cuando digo ficheros de HTML, CSS y JavaScript es por sencillez. Ya sabemos que las páginas web se pueden hacer de varias formas. Tú puedes embeber CSS en un fichero HTML, o JavaScript. O pueden ir por separado. Y por supuesto, contener imágenes, sonidos, etc.

Cada **mensaje** HTTP, ya sea petición o respuesta, está dividido en dos partes:

- Una **cabecera** con metadatos. La cabecera es obligatoria, pero sus campos no tienen por qué serlo.
- Un **cuerpo** del mensaje, opcional. Típicamente tiene los datos que se intercambian entre el Cliente y Servidor. A veces simplemente es necesario responder al cliente con un 'Ok', por lo que no hay cuerpo.

Petición HTTP

Una **petición HTTP** (request) normalmente no la puedes ver, se trata de algo interno del navegador. Sí es posible verlas con diferentes programas de desarrollo. Esto es un ejemplo de **petición HTTP**:

```
GET /index.html HTTP/1.1
Host: www.ejemplo.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: es-ES,es;q=0.9
Accept-Encoding: gzip, deflate
Connection: keep-alive
```

Respuesta HTTP

Las **respuestas HTTP** (response) son generadas por el servidor. Incluyen la versión del HTTP, un **código de estado**, y la propia respuesta del servidor normalmente en el **cuerpo** del mensaje. Es en este cuerpo dónde encontramos el HTML, CSS y JavaScript.

Un ejemplo de **respuesta HTTP**:

```
HTTP/1.1 200 OK
Date: Mon, 10 Nov 2025 16:55:00 GMT
Server: Apache/2.4.54 (Ubuntu)
Content-Type: text/html; charset=UTF-8
Content-Length: 137

<!DOCTYPE html>
<html>
<head><title>Ejemplo</title></head>
<body><h1>¡Hola desde el servidor!</h1></body>
</html>
```

¿Te has dado cuenta cómo hay una página web con HTML en el cuerpo de la respuesta?

Pues efectivamente: esto es lo que renderiza el Navegador

Métodos HTTP

Cuando lanzamos una **petición HTTP**, se tiene que indicar el **método** o la operación que intentamos ejecutar. Existen varios **métodos** diferentes. Los más usadas en entorno web, son:

- **GET**: Solicitan una página web. No suelen tener cuerpo.
- **POST**: Envía datos de un formulario de una página web al servidor. Suelen tener cuerpo.

Pero hay más, que no son tan usados en entorno web sino en Servicios Web:

- **PUT**: Se usa para actualizar un recurso. Suelen tener cuerpo.
- **DELETE**: Se usan para borrar un recurso. A veces tienen cuerpo.
- **HEAD**: Equivale a un GET, pero solo devuelve cabeceras.
- Etc.

Código de estado HTTP

Las **respuestas HTTP** tienen siempre un **código de estado**, independientemente de si hay un cuerpo o no. Estos códigos son números que indican lo que ha pasado en el servidor al procesar la petición.

- **Códigos con formato 1xx:** Respuestas informativas. Indica que la petición ha sido recibida y se está procesando.
- **Códigos con formato 2xx:** Respuestas correctas. Indica que la petición ha sido procesada correctamente.
- **Códigos con formato 3xx:** Respuestas de redirección. Indica que el cliente necesita realizar más acciones para finalizar la petición.
- **Códigos con formato 4xx:** Errores causados por el cliente. Indica que ha habido un error en la petición porque el cliente ha hecho algo mal.
- **Códigos con formato 5xx:** Errores causados por el servidor. Indica que ha habido un error en la petición a causa de un fallo en el servidor.

Varios de los códigos de respuesta más comunes son:

- **100 Continue.** Todo hasta ahora está bien y el cliente debe continuar con la solicitud o ignorarla
- **200 OK.** La solicitud ha tenido éxito.
- **201 Created.** Se ha creado un nuevo recurso. Respuesta típica de PUT
- **301 Moved Permanently.** La URI del recurso ha cambiado.
- **400 Bad Request.** El server no entiende, sintaxis inválida
- **401 Unauthorized.** Tienes que autenticarte.
- **403 Forbidden.** No tienes permiso de acceso
- **404 Not Found.** No podemos encontrar el recurso.
- **500 Internal Server Error.** Error en el Servidor

Importante – ¿Es por eso que me sale 404 en el Navegador?

Efectivamente. Cuando pulsas un enlace web (URL) y el servidor no tiene esa página (recurso), lo que hace es generar una respuesta HTTP con el cuerpo vacío y el **código de estado 404**.

Para temas de desarrollo web de este módulo, no tienes que preocuparte demasiado. Basta con que entiendas lo que significan, porque los generan los servidores.

1.5 API Fetch

Fetch es la API nativa e integrable de JavaScript moderno para realizar peticiones HTTP asíncronas basadas en **Promesas**. Reemplazó al antiguo e incómodo objeto XMLHttpRequest.

Es decir: es una parte de código JavaScript que me permite hacer una petición HTTP ‘en paralelo’ mientras estoy viendo una página web cargada en mi navegador, sin tener que volver a submitir la página entera. Así podíamos pulsar un botón y refrescar una tabla con nuestros clientes de forma automática.

Esto lo veremos más adelante. Por ahora, que te suene.

1.6 Programando en el Lado Cliente

Entonces... ¿qué se programa exactamente en el Lado Cliente?

Cuando lanzamos una URL y solicitamos una página web, lo que nos llega es en el cuerpo de la **respuesta HTTP** es **HTML**, **CSS** y **JavaScript**. De estas tres partes, solamente una tiene código ejecutable: el **JavaScript**. Por tanto, lo que el navegador puede ejecutar es eso: JavaScript.

Por favor, entiende que estamos simplificando las cosas.

El Navegador te muestra la página, con su contenido, sus colorines, etc. Entonces, vas al formulario para introducir el DNI, y te equivocas. ¿Cómo lo sabes? Porque un código JavaScript se ha lanzado a ejecución en el navegador para verificar que ese DNI sea correcto. Al dar error, te ha puesto en rojo el inputText.

JavaScript permite hacer cuatro cosas con una página web cargada en el Navegador:

- Modificar la estructura HTML (**DOM**) de la página, o modificar los estilos CSS (**CSSOM**) de la página, de forma **temporal**.
- Reaccionar a los eventos del usuario: clicks de botón, etc.
- Comunicación Fetch en segundo plano (API Fetch)
- Validar los datos antes de enviarlos, hacer cálculos (lógica) y guardarlos de forma local o en **Cookies**

Importante – ¿Por qué dices temporalmente?

Cuando cambiamos el HTML o CSS de una página cargada en nuestro Navegador, por ejemplo, cambiando el color de un texto a rojo, no estamos modificando nada realmente. Si solicitamos la página de nuevo al Servidor, nos devolverá el texto en negro.

1.7 Disciplinas transversales

... pero no basta con programar. Hay que saber **programar bien**.

Cuando hablamos del Desarrollo Web en Entorno Cliente, hay cuatro cosas que diferencian al 'chapuzas' del buen programador. Obviamente, es **imposible** adquirir todas estas competencias de golpe. Las trabajaremos a lo largo del módulo.

Son las siguientes:

- **Accesibilidad Web (a11y - Accessibility)**: Garantiza que cualquier persona, independientemente de sus capacidades físicas, sensoriales o cognitivas (o del dispositivo que use), pueda navegar e interactuar con la aplicación. El **estándar** son las Pautas **WCAG** (Web Content Accessibility Guidelines) de la W3C.
- **Rendimiento en Cliente**: Mide la rapidez con la que una página web se carga, se renderiza y responde a la interacción del usuario. Las **Core Web Vitals** de Google son una serie de métricas que nos permite evaluar este rendimiento.
- **Seguridad desde el Entorno Cliente**: Como norma, **el entorno cliente es inseguro** por naturaleza. Pero, hay formas de blindarlo contra los ataques más comunes. Veremos alguna forma de conseguirlo.
- **SEO desde el Cliente**: Se enfoca en facilitar que los motores de búsqueda (como el Bot de Google) puedan indexar la estructura y el contenido de la aplicación web.

Para nosotros, esto se traduce en que seguiremos unas **buenas prácticas** desde el inicio:

- Accesibilidad desde el inicio:
 - **HTML semántico**: HTML5 ofrece una serie de etiquetas con un significado caro para el motor de renderizado. Hay que usarlas. No abusar del <div>
 - **Contraste visual**: un contraste de colores adecuado no solo beneficia a personas con baja visión, daltonismo o presbicia; también asegura la legibilidad en pantallas de gama baja.
 - **Foco visible**: Se trata del indicador gráfico que señala qué elemento interactivo de la pantalla (botón, enlace, campo de texto, etc.) tiene el control del teclado en cada momento.
- Código legible y consistente:
 - Convenio de nombres de JavaScript, formato, etc.
 - Uso de Herramientas de **Linting** y Formateo.
- Estructura mínima de cada proyecto que hagamos:
 - Carpetas: src/, assets/, index.html.
- Separación de responsabilidades:
 - Por un lado, el contenido: HTML
 - Por otro lado, los estilos: CSS
 - Por otro lado, el comportamiento: JavaScript
- Documentación breve en el README con instrucciones de ejecución.

1.8 Interoperabilidad

La **Interoperabilidad Web**: Es la capacidad de que una misma aplicación o sitio web funcione de manera uniforme, predecible y sin errores en cualquier navegador, sistema operativo y dispositivo, sin necesidad de escribir código específico o adaptaciones exclusivas para cada uno.

De nuevo, ChatGPT en acción.

Una parte muy importante de la Web es la interoperabilidad. Una página web tiene que verse igual y funcionar igual en cualquier navegador sin tener que andar perdiendo el tiempo con parches.

Esto es muy importante. Cuando Internet era nueva, nadie pudo prever su crecimiento exponencial. Al principio era una amalgama de redes más pequeñas unidas con cinta aislante y buenas intenciones. Cuando dos redes se encontraban, era posible 'hacer un apaño' para que todo funcionase. Pronto esto fue imposible. Ahí es cuando aparecieron los **estándares**. Si querías unirte a Internet, no te quedaba otra que seguir esos estándares o te quedabas fuera. A día de hoy, podemos decir que todo Internet funciona de la misma manera gracias a eso.

Esta filosofía se llevó un paso más allá. Ya no bastaba con cumplir los estándares y que tu página funcione, ahora es necesario programar de manera que asegures la **Interoperabilidad** de tu web.

Los organismos que se encargan de los temas de estandarización son:

- **WHATWG** (Web Hypertext Application Technology Working Group)
Se encargan de mantener el estándar de **HTML** y las APIs del **DOM**. Antaño de esto se encargaba la W3C, y había versiones cerradas de HTML (la 4, la 5...) Ahora, es un estándar vivo que va evolucionando.

- **W3C** (World Wide Web Consortium)

Define las especificaciones de **CSS**, las pautas de accesibilidad (**WCAG**), **SVG** y arquitecturas de red.

- **ECMA International**

Mantiene la especificación **ECMAScript**, que es el estándar oficial bajo el cual se define el lenguaje **JavaScript**. El ECMAScript en realidad son sólo las especificaciones teóricas, mientras que JavaScript son las implementaciones prácticas.

Existen diversas herramientas para asegurarse la Interoperabilidad. Por ejemplo

- **caniuse.com**: Esta web permite saber si un elemento puede utilizarse en una versión de un navegador, entre otras cosas. Prueba a entrar y busca <div>. Mira lo que ocurre.
- **webstatus.dev**: Esta web muestra los resultados de miles de test para cerrar brechas de compatibilidad entre navegadores.
- **Plugins: VS Code** dispone de plugins como slint-plugin-compat para analizar tu JS mientras escribes. Es una herramienta de **Linting**.
- **a11ysupport.io**: Esta web es el "Can I Use" de la accesibilidad.

Ejercicio 1 - DevTools

Las **DevTools** (Developer Tools) son las utilidades integradas en los navegadores web modernos (Chrome, Firefox, Edge, Safari) que permiten inspeccionar, depurar, auditar y optimizar aplicaciones web en tiempo real.

Es decir: nos permiten ‘ver’ el HTML, CSS y JavaScript de una página del navegador, modificarlo y ver los cambios que se producen como consecuencia de los cambios. Estos cambios, por supuesto, son temporales.

Ni qué decir tiene que las vas a usar mucho a lo largo de todo el módulo, y es posible que ya las conozcas. Para utilizarlas, basta con pulsar **F12** (o Ctrl + Shift + I).

Pasos:

1. Abre **DevTools.html** en tu navegador.
2. Inspecciona la página.
3. Abre las **DevTools**.
 - a. Consulta el panel de **elementos HTML**. Te muestra la estructura de la página. (DOM)
 - b. Consulta el panel de **estilos**. Te muestra los estilos CSS que se aplican para los elementos HTML seleccionado en el panel anterior.
 - c. Consulta el panel de la **consola**. Te muestra la salida del código JavaScript.
4. Estas son las funcionalidades mínimas. Investiga el resto de las DevTools por tu cuenta. Debería de sonarte con lo que hemos visto este tema.
5. Realiza los cambios especificados a continuación.

Tareas:

1. Inspecciona desde el DOM la cabecera de la página y cambia el texto.
2. Refresca con F5 la página en el navegador. ¿Qué ha sucedido? ¿Por qué?
3. Observa la imagen del círculo azul con el texto ‘DEV’. Localízalo en la pestaña Network, y consulta su enlace y tamaño. Verás que la imagen se obtiene <http://www.w3.org/2000/svg>
4. Pulsa el botón “Ejecutar Acción”. Observa las salidas por consola. Observa el código JavaScript en la página. Esta es una manera simple de comprobar que el código funciona bien.
5. Auditoría con **Lighthouse**. Entra en la url de tu centro educativo (no funciona para páginas en local). A continuación, sigue estos pasos:
 - a. Abre la web en una ventana de incógnito, esto evita problemas con otras extensiones instaladas.
 - b. Abre las DevTools.
 - c. Selecciona la pestaña **Lighthouse**.
 - d. Configura la auditoría:
 - i. **Modo:** Navigation (Default).
 - ii. **Dispositivo:** Elegir entre Mobile o Desktop.
 - iii. **Categorías:** Seleccionar las áreas a auditar (Rendimiento, Accesibilidad, Buenas Prácticas, SEO).
 - e. Dale a **Analyze page load**
 - f. Comprueba los resultados.

Ejercicio 2 - Accesibilidad express (A11Y)

Vamos a trabajar ahora temas de accesibilidad.

Pasos:

1. Abre **Accesibilidad.html** en tu navegador.
2. Inspecciona la página.
3. Esta página tiene **errores** de accesibilidad, aunque tú no te des cuenta.
4. Abre las **DevTools**.
5. Realiza los arreglos especificados a continuación

Tareas:

1. HTML Semántico, Headings, Alt y Label – Existen los siguientes errores en la página.
 - a. Jerarquía de Headings: No se sigue correctamente la progresión <H1>, <H2>, <H3>...
 - b. La imagen carece de atributo alt.
 - c. El campo input carece de label e id asociada.
2. Contraste. El texto secundario que te solicita rellenar los campos tiene el contraste mal puesto. ¿Texto gris claro sobre fondo blanco? Existe una fórmula oficial de contraste de color (WCAG 2.1 / 2.2). Puedes comprobar si el contraste es correcto con Lighthouse u otro plugin.

Otra forma de comprobar las ratios de color es yendo al CSS, buscar el color y pinchar en el cuadradito. Ahí te permitirá ver las ratios y cambiarlos.

3. Pulsa Tabulador. Verás que se selecciona el input. Pulsa de nuevo. ¿Dónde está el foco? Debería de estar en el botón de registrase.

Este error es de los estilos. Elimina `button:focus {}` y añade uno nuevo.